

Facultad de Ciencias

MANEJO DE DATOS

Buenas Prácticas, POO y Principios SOLID en Python

Tarea 01

Autores:

García Galindo Melissa

Martínez Molina Karen

Montoya Vazquez Ángel Eduardo

Vázquez García Mauricio Alexis

8 de Septiembre del 2026

1. Investigación

1.1. “Parte A — PEP 8

PEP 8 (Python Enhancement Proposal 8) se refiere a una guía de convenciones estilísticas a seguir para escribir código en python, que van desde conocer el tamaño máximo para las líneas de código, propuestas en cómo nombrar a las variables, volver el código legible, entre otras cosas. Incluso existen linters (herramientas que analizan el código de forma automática para detectar errores o malas prácticas) y auto formatters (herramientas que pueden reescribir código o indicar nuestros errores) que podemos usar en caso de no recordar todas las convenciones al escribir código.

A continuación colocamos un ejemplo donde se pueden visualizar las aplicaciones de estas convenciones al código:

Regla 1: Sangría con 4 espacios por nivel

Esta regla nos dice que debemos usar 4 espacios por nivel de sangría. Los espacios son el método preferido de sangría; los tabuladores solo deben usarse para mantener consistencia con código que ya los utilice. Recordando que Python prohíbe mezclar tabuladores y espacios.

Antes (Incorrecto):

```
def calcular_prima(monto):  
    # Uso incorrecto de 2 espacios de sangría  
    factor = 1.16  
    return monto * factor
```

Después (Correcto):

```
def calcular_prima(monto):  
    # Sangría estándar de exactamente 4 espacios  
    factor = 1.16  
    return monto * factor
```

Regla 2: Longitud máxima de línea

Python limita todas las líneas a un máximo de 79 caracteres. Sin embargo, para bloques fluidos de texto largo (cadenas de documentos o comentarios), el límite es de 72 caracteres y el mecanismo preferido para envolver líneas largas es la continuación implícita dentro de paréntesis, corchetes o llaves.

Antes (Incorrecto - Línea continua de 118 caracteres):

```
resultado = registrar_asegurado(nombre_cliente, edad_cliente, factor_riesgo, suma_asegurada_mxn, fecha_emision_poliza)
```

Después (Correcto - Formato vertical con paréntesis):

```
resultado = registrar_asegurado(  
    nombre_cliente,  
    edad_cliente,  
    factor_riesgo,  
    suma_asegurada_mxn,  
    fecha_emision_poliza,  
)
```

Regla 3: Líneas en blanco

Esta regla nos habla sobre rodear las funciones y definiciones de clases de nivel superior con dos líneas en blanco y los métodos dentro de una clase se separan entre sí con una sola línea en blanco.

Antes (Incorrecto - Bloques principales pegados):

```
class Poliza:
    def __init__(self, monto):
        self.monto = monto
    def procesar(self):
        return self.monto * 1.10
def ejecutar():
    p = Poliza(1000)
```

Después (Correcto - Una línea entre métodos y dos antes de la función externa):

```
class Poliza:
    def __init__(self, monto):
        self.monto = monto

    def procesar(self):
        return self.monto * 1.10

def ejecutar():
    p = Poliza(1000)
```

Regla 4: Importaciones

Las importaciones deben estar en líneas separadas y colocarse al inicio del archivo, también que deben agruparse en tres bloques separados por una línea en blanco: 1) Biblioteca estándar, 2) Terceros relacionados, y 3) Módulos locales.

Antes (Incorrecto):

```
import os, sys, requests, math
```

Después (Correcto):

```
import math
import os
import sys

import requests
```

Regla 5: Nombres dunder a nivel de módulo

Los atributos a nivel de módulo con nombres "dunder" deben ubicarse después de la cadena de documentos del módulo, pero antes de cualquier sentencia de importación, excepto si existen importaciones desde `__future__`.

Antes (Incorrecto):

```
"""Módulo de cálculo de primas."""

import math
import os

__all__ = ["CalculadoraPrima"]
__author__ = "Melissa García"
__version__ = "1.0.0"
```

Después (Correcto):

```
"""Módulo de cálculo de primas."""

from __future__ import annotations

__all__ = ["CalculadoraPrima"]
__author__ = "Melissa García"
__version__ = "1.0.0"

import math
import os
```

Regla 6: Citas de cadena

Esta regla indica que en Python, las cadenas con comillas simples y comillas dobles son idénticas. PEP 8 no impone una recomendación sobre cuál de las dos usar; la regla fundamental es elegir una convención y mantenerla de forma consistente en todo el archivo. Sin embargo, la única excepción ocurre cuando una cadena contiene comillas por dentro: se debe usar el otro tipo de comilla en el exterior para evitar el uso de barras invertidas de escape (`\`), mejorando así la legibilidad.

Antes (Incorrecto - mezclar arbitrariamente y usar escapes innecesarios):

```
# Inconsistencia de estilo y uso evitable de escape \
mensaje_uno = 'Bienvenido a la aseguradora'
mensaje_dos = "Procesando cálculo de póliza"
mensaje_tres = 'El asegurado respondió: \'Si\' a la pregunta'
```

Después (Correcto - estilo homogéneo y comillas dobles exteriores):

```
# Convención uniforme y sin caracteres de escape innecesarios
mensaje_uno = 'Bienvenido a la aseguradora'
mensaje_dos = 'Procesando cálculo de póliza'
mensaje_tres = "El asegurado respondió: 'Si' a la pregunta"
```

Regla 7: Espacios en blanco en expresiones y enunciados

Esta regla dice que hay que evitar espacios en blanco superfluos inmediatamente dentro de paréntesis, corchetes o llaves. Se deben evitar espacios inmediatamente antes de una coma, punto y coma o dos puntos. En rebanados o cortes de secuencias (slices), los dos puntos actúan como un operador binario y deben llevar la misma cantidad de espacio a ambos lados (preferentemente ninguno si los parámetros son expresiones simples).

Antes (Incorrecto - huecos internos y espacios previos a comas):

```
# Delimitadores con huecos internos, espacios previos a comas y slices irregulares
edades = [ 18, 25, 45 , 65 ]
rango_edades = edades[ 1 : 3 ]
datos_asegurado = ( 'M' , 500000.0 )
factores = { 'F' : 1.5 , 'M' : 2.0 }
```

Después (Correcto - delimitadores pegados y espaciado consistente):

```
# Delimitadores limpios, puntuación estándar y espaciado consistente
edades = [18, 25, 45, 65]
rango_edades = edades[1:3]
datos_asegurado = ('M', 500000.0)
factores = {'F': 1.5, 'M': 2.0}
```

Regla 8: Comentarios en bloque

Esta regla explica que los comentarios en bloque aplican a todo o parte del código que les sigue, y deben mantener la misma sangría que dicho bloque de código. Cada línea de un comentario en bloque debe comenzar con un # seguido de exactamente un espacio en blanco. Los párrafos se separan por una línea vacía que contenga únicamente el carácter #. Deben redactarse como oraciones completas y actualizarse cuando el código cambie.

Antes (Incorrecto - sin sangría, sin espacio tras el numeral):

```
def aplicar_ajuste_edad(edad, sexo):
#Comentario sin sangrar y pegado al margen izquierdo
#sin espacio tras el numeral
    edad_final = edad
    if sexo == 'F':
        #resta 5 anos (Comentario desactualizado: el codigo resta 10)
        edad_final -= 10
    return edad_final
```

Después (Correcto - sangrado correcto, separación de párrafos):

```
def aplicar_ajuste_edad(edad, sexo):
    # Aplica las deducciones correspondientes al perfil demográfico.
    #
    # Para el sexo femenino, la normativa establece una reducción
    # directa de 10 años sobre la edad base del asegurado.
    edad_final = edad
    if sexo == 'F':
        edad_final -= 10
    return edad_final
```

Regla 9: Cadenas de documentación (Docstrings)

Esta regla indica que se deben escribir docstrings para todos los módulos, funciones, clases y métodos públicos. Deben delimitarse siempre con comillas triples dobles " (según PEP 257) y, para docstrings multilínea, las comillas de cierre deben colocarse en una línea independiente.

Antes (Incorrecto - comentarios simples con # en lugar de docstring):

```
def cotizar_prima(monto, factor):  
    # Calcula la prima multiplicando monto por factor y dividiendo entre mil  
    # Regresa un numero flotante  
    return (monto * factor) / 1000
```

Después (Correcto - docstring formal con comillas triples):

```
def cotizar_prima(monto, factor):  
    """Calcula la prima anual de la póliza de seguro.  
  
    Multiplica la suma asegurada por el factor actuarial de edad  
    y divide el resultado entre mil según la fórmula oficial.  
    """  
    return (monto * factor) / 1000
```

Regla 10: Convenciones de nombres para clases, funciones y constantes

Los nombres de clases deben utilizar la convención CapWords (PascalCase). Los nombres de funciones y variables deben ir en minúsculas, separando palabras con guiones bajos (snake_case). Las constantes deben definirse a nivel de módulo en letras mayúsculas separadas por guiones bajos (UPPER_SNAKE_CASE).

Antes (Incorrecto - constante minúscula, clase snake_case, método camelCase):

```
tasa_cambio = 21.13  
  
class factor_calculo:  
    def obtenerValor(self, edad_actual):  
        return edad_actual * 1.5
```

Después (Correcto - constante mayúsculas, clase CapWords, método snake_case):

```
TASA_CAMBIO = 21.13  
  
class FactorCalculo:  
    def obtener_valor(self, edad_actual):  
        return edad_actual * 1.5
```

2. Principios SOLID

2.1. Parte B — ¿Qué es SOLID?

SOLID son las siglas de los 5 principios de la programación orientada a objetos, básicamente son consejos y buenas prácticas de desarrollo para tener un código mantenible, flexible, y fácil de probar.

S: Single Responsibility Principle (Principio de Responsabilidad Única)

Según este principio “una clase debería tener una, y solo una, razón para cambiar”. Nos ayuda porque cuando algo no funciona, sabes exactamente a qué archivo o clase ir a revisar, en lugar de buscar entre un buen de líneas de código revuelto.

Viola el principio:

```
class Factura:
    def __init__(self, monto):
        self.monto = monto

    def calcular_total(self):
        # Calcula impuestos y total
        return self.monto * 1.16

    def guardar_en_base_datos(self):
        # Conecta a SQL y guarda la factura
        print("Guardando factura en la base de datos...")

    def enviar_correo(self):
        # Configura el servidor SMTP y envía el correo
        print("Enviando correo al cliente...")
```

Corregido siguiendo el principio S:

```
class Factura:
    """Responsabilidad: representar los datos de la factura y su cálculo."""
    def __init__(self, monto):
        self.monto = monto

    def calcular_total(self):
        return self.monto * 1.16

class FacturaRepositorio:
    """Responsabilidad: gestionar la persistencia en la base de datos."""
    def guardar(self, factura):
        print(f"Guardando factura por ${factura.calcular_total()}...")

class ServicioNotificacion:
    """Responsabilidad: gestionar el envío de correos y avisos."""
    def enviar_correo(self, factura, destinatario):
        print(f"Enviando correo con la factura a {destinatario}...")
```

O: Open Closed Principle (Principio de Apertura y Cierre)

Este principio en palabras sencillas, nos indica que el código debe estar abierto para que le agregues cosas nuevas, pero cerrado para que no tengas que modificar lo que ya funciona.

Viola al principio O:

```
class CalculadoraDescuento:
    """Calcula el descuento aplicable según el tipo de cliente."""

    def calcular_descuento(self, tipo_cliente, monto):
        if tipo_cliente == "Regular":
            return monto * 0.05
        elif tipo_cliente == "Premium":
            return monto * 0.15
        elif tipo_cliente == "Estudiante":
            return monto * 0.20
        else:
            return 0.0
```

No está cerrado a modificaciones: Cada vez que el negocio inventa una nueva categoría te ves obligado a abrir y modificar la clase, añadiendo más bloques.

Corregido siguiendo el principio O:

```
class Descuento:
    """Clase base de la que heredan todos los descuentos."""

    def calcular(self, monto):
        return 0.0

class DescuentoRegular(Descuento):
    """Aplica el 5% de descuento."""

    def calcular(self, monto):
        return monto * 0.05

class DescuentoPremium(Descuento):
    """Aplica el 15% de descuento."""

    def calcular(self, monto):
        return monto * 0.15

class DescuentoEstudiante(Descuento):
    """Aplica el 20% de descuento."""

    def calcular(self, monto):
        return monto * 0.20

class CalculadoraDescuento:
    """Clase principal cerrada a modificaciones."""

    def calcular_total_descuento(self, tipo_descuento, monto):
        # Llama al método 'calcular' del objeto que le mandes
        return tipo_descuento.calcular(monto)
```

Sí estamos respetando el principio O porque está cerrado a modificaciones “CalculadoraDescuento” no tiene ningún if ni elif. No tienes que volver a editarla jamás y está abierto a extensiones si mañana agregan descuento de empleado, solo creas una clase nueva que herede de Descuento.

L: Liskov Substitution Principle (Principio de Sustitución de Liskov)

Según este principio funciona porque si una clase hija hereda de una clase padre, deberías poder usar a la hija en cualquier lugar donde usabas al padre sin que el programa tenga problemas.

Viola al principio L:

```
class CuentaBancaria:
    """Clase base para todas las cuentas."""

    def __init__(self, saldo):
        self.saldo = saldo

    def retirar(self, monto):
        if monto > self.saldo:
            raise ValueError("Saldo insuficiente.")
        self.saldo -= monto
        return self.saldo

class CuentaPlazoFijo(CuentaBancaria):
    """Cuenta de inversión a plazo fijo donde el dinero está bloqueado."""

    def retirar(self, monto):
        # Viola Liskov: rompe el comportamiento esperado lanzando un error
        raise Exception("No se puede retirar dinero de una cuenta a plazo fijo.")
```

Si tienes una función del sistema pensada para aplicar retiros a una lista de cuentas, entonces el programa colapsa porque CuentaPlazoFijo no puede sustituir a su clase base CuentaBancaria sin alterar la estabilidad esperada, por eso viola al principio L.

Corregido siguiendo el principio L:

```
class CuentaBancaria:
    """Clase base con operaciones comunes a cualquier cuenta."""

    def __init__(self, saldo):
        self.saldo = saldo

    def obtener_saldo(self):
        return self.saldo

class CuentaRetirable(CuentaBancaria):
    """Subclase específica para cuentas que sí permiten retiros."""

    def retirar(self, monto):
        if monto > self.saldo:
            raise ValueError("Saldo insuficiente.")
        self.saldo -= monto
        return self.saldo

class CuentaAhorro(CuentaRetirable):
    """Cuenta de ahorro estándar que permite retiros."""
    pass

class CuentaPlazoFijo(CuentaBancaria):
    """Cuenta de inversión: hereda saldo pero no la capacidad de retiro."""
    pass
```

Ahora la función que realiza retiros solo exige cuentas de tipo CuentaRetirable, garantizando que cualquier subclase que reciba pueda operar sin errores inesperados y tanto CuentaAhorro como cualquier otra futura cuenta con retiros pueden sustituir a CuentaRetirable sin que la función falle ni requiera validaciones manuales de tipo. Entonces este código si está siguiendo la propiedad L.

I: Interface Segregation Principle (Principio de Segregación de Interfaces)

Según esta propiedad, “Haz interfaces que sean específicas para un tipo de cliente”. Prácticamente esta propiedad nos está diciendo que no obliguemos a nadie a cargar con cosas que realmente no va a usar, es mejor o preferible estructurar interfaces reducidas en lugar de una interfaz general saturada.

Viola al principio I:

```
class DispositivoInteligente:
    """Interfaz sobrecargada que asume que todo dispositivo hace de todo."""

    def encender(self):
        pass

    def reproducir_musica(self):
        pass

    def ajustar_temperatura(self, grados):
        pass

class BocinaBluetooth(DispositivoInteligente):
    """Solo reproduce sonido, pero se ve forzada a cargar con métodos de clima."""

    def encender(self):
        print("Bocina encendida y vinculada.")

    def reproducir_musica(self):
        print("Reproduciendo playlist en Spotify...")

    def ajustar_temperatura(self, grados):
        # Viola el Principio I: una bocina no regula clima,
        # pero está obligada a implementarlo para cumplir con la clase padre.
        pass
```

Lo viola porque DispositivoInteligente es una interfaz sobrecargada: Metió en una sola bolsa funciones que pertenecen a aparatos totalmente distintos.

Corregido siguiendo propiedad I:

```
class Encendible:
    """Contrato específico para aparatos que se pueden encender o apagar."""

    def encender(self):
        pass

class ReproductorAudio:
    """Contrato específico para dispositivos que emiten sonido."""

    def reproducir_musica(self):
        pass

class ControladorClima:
    """Contrato específico para regular la temperatura."""

    def ajustar_temperatura(self, grados):
        pass
```

En este código ni la bocina ni el termostato tienen métodos vacíos con pass, además si cambias la forma en que funciona ajustar_temperatura, la clase BocinaBluetooth ni se entera ni corre riesgo de romperse y por último usas herencia múltiple para combinar solo las piezas que el objeto realmente sabe y necesita hacer. Entonces estamos siguiendo el principio I de manera correcta.

D: Dependency Inversion Principle (Principio de Inversión de la Dependencia)

Este es el último elemento de los principios de SOLID, y dice que “Depende de abstracciones, no de clases concretas”. Este principio la podemos ver como que las partes importantes de tu programa no deben depender de los detalles técnicos, sino de acuerdos generales.

Viola al principio D:

```
class ServicioSMS:
    """Detalle de bajo nivel: envío mediante mensajes SMS."""

    def enviar_sms(self, mensaje):
        print(f"Enviando SMS al teléfono: {mensaje}")

class GestorNotificaciones:
    """Módulo de alto nivel: orquesta las alertas del sistema."""

    def __init__(self):
        # ERROR: Se acopla rígidamente a una herramienta concreta
        self.servicio = ServicioSMS()

    def notificar_usuario(self, texto):
        self.servicio.enviar_sms(texto)
```

Este código está mal porque la clase de alto nivel (GestorNotificaciones) está amarrada rígidamente al detalle técnico al crear directamente `self.servicio = ServicioSMS()` en su constructor; esto invierte el flujo correcto del diseño, obligándote a modificar y reescribir la lógica principal del sistema cada vez que quieras cambiar el canal de mensajería (como sustituir SMS por correo o WhatsApp) en lugar de simplemente permitir que cualquier servicio compatible se conecte desde afuera de forma flexible.

Corregido siguiendo el principio D:

```
class CanalNotificacion:
    """Acuerdo o base común para cualquier medio de envío."""

    def enviar(self, mensaje):
        pass

class ServicioSMS(CanalNotificacion):
    """Implementación concreta mediante SMS."""

    def enviar(self, mensaje):
        print(f"Enviando SMS al teléfono: {mensaje}")

class ServicioEmail(CanalNotificacion):
    """Implementación concreta mediante correo electrónico."""

    def enviar(self, mensaje):
        print(f"Enviando correo al buzón: {mensaje}")

class GestorNotificaciones:
    """Módulo de alto nivel desacoplado de las herramientas concretas."""

    def __init__(self, canal: CanalNotificacion):
        # Inyección de dependencia: recibe el canal desde afuera
        self.canal = canal

    def notificar_usuario(self, texto):
        self.canal.enviar(texto)
```

Aquí si estamos siguiendo la propiedad D, está bien porque GestorNotificaciones ya no construye sus propias herramientas ni sabe qué tecnología hay detrás, sino que recibe cualquier servicio compatible desde afuera a través de su constructor; así, la lógica principal depende únicamente del acuerdo general (enviar) y puedes cambiar de SMS a correo o WhatsApp sin tener que tocar ni una sola línea del gestor.